

Asignación: Query Methods y Consultas en Spring Data JPA

Curso: Computación en Internet II (Compunet II) -- Universidad ICESI

Instrucciones: Para cada ejercicio:

1. Declara el método de consulta en la interfaz del **Repositorio**.
 2. Implementa un método en el **Service** que inyecte el repositorio e invoque la consulta.
-

Ejercicio 1: Buscar usuario por correo

- **Requerimiento:** Buscar un usuario por su `correoInstitucional` exacto.
- **Repositorio:** `UsuarioRepository`
- **Service:** `UsuarioService`

```
// Repositorio:  
  
// Service:
```

Ejercicio 2: Verificar existencia de correo

- **Requerimiento:** Comprobar si ya existe un usuario con un `correoInstitucional` determinado (retorna `boolean`).
- **Repositorio:** `UsuarioRepository`
- **Service:** `UsuarioService`

```
// Repositorio:  
  
// Service:
```

Ejercicio 3: Profesores activos por departamento

- **Requerimiento:** Obtener los profesores activos (`active = true`) que pertenezcan a un `departamento` específico, sin distinguir mayúsculas de minúsculas.
- **Repositorio:** `ProfesorRepository`
- **Service:** `ProfesorService`

```
// Repositorio:
```

```
// Service:
```

Ejercicio 4: Cursos por rango de créditos

- **Requerimiento:** Obtener los cursos cuya cantidad de **creditos** se encuentre dentro de un rango (**min** y **max**).
- **Repositorio:** **CursoRepository**
- **Service:** **CursoService**

```
// Repositorio:
```

```
// Service:
```

Ejercicio 5: Buscar cursos por coincidencia en el nombre

- **Requerimiento:** Buscar cursos cuyo **nombre** contenga un texto dado, sin distinguir mayúsculas de minúsculas.
- **Repositorio:** **CursoRepository**
- **Service:** **CursoService**

```
// Repositorio:
```

```
// Service:
```

Ejercicio 6: Profesores por especialidad ordenados por apellido

- **Requerimiento:** Obtener los profesores de una **especialidad** dada (sin distinguir mayúsculas) ordenados alfabéticamente por su **apellido** de forma ascendente.
- **Repositorio:** **ProfesorRepository**
- **Service:** **ProfesorService**

```
// Repositorio:
```

```
// Service:
```

Ejercicio 7: Estudiantes por dominio de correo

- **Requerimiento:** Obtener los estudiantes cuyo **correoInstitucional** termine con una cadena o dominio dado (ej. "@icesi.edu.co"), ignorando mayúsculas/minúsculas.
- **Repositorio:** **EstudianteRepository**
- **Service:** **EstudianteService**

```
// Repositorio:
```

```
// Service:
```

Ejercicio 8: Conteo de estudiantes activos

- **Requerimiento:** Contar el total de estudiantes cuyo estado sea activo (**active = true**).
- **Repositorio:** **EstudianteRepository**
- **Service:** **EstudianteService**

```
// Repositorio:
```

```
// Service:
```

Ejercicio 9: Cursos asignados a un profesor (ManyToOne)

- **Requerimiento:** Obtener todos los cursos que dicta un profesor a partir del **id** del profesor.
- **Repositorio:** **CursoRepository**
- **Service:** **CursoService**

```
// Repositorio:
```

```
// Service:
```

Ejercicio 10: Cursos según el departamento del profesor

- **Requerimiento:** Obtener los cursos cuyo profesor pertenezca a un **departamento** determinado (navegando la relación **profesor**), sin distinguir mayúsculas.
- **Repositorio:** **CursoRepository**
- **Service:** **CursoService**

```
// Repositorio:
```

```
// Service:
```

Ejercicio 11: Usuarios activos por nombre de rol (ManyToMany)

- **Requerimiento:** Obtener todos los usuarios activos (**active = true**) que tengan asignado un rol con un **nombre** dado (navegando la relación **roles**), sin distinguir mayúsculas.
- **Repositorio:** **UsuarioRepository**
- **Service:** **UsuarioService**

```
// Repositorio:
```

```
// Service:
```

Ejercicio 12: Verificar matrícula en tabla intermedia

- **Requerimiento:** Comprobar si existe un registro de matrícula para un **estudianteId** y un **cursoId** dados en **EstudianteCurso** (retorna **boolean**).
- **Repositorio:** **EstudianteCursoRepository**
- **Service:** **MatriculaService**

```
// Repositorio:
```

```
// Service:
```

Ejercicio 13: Cursos por créditos mínimos ordenados descendentemente

- **Requerimiento:** Obtener los cursos con **creditos** mayores o iguales a un valor mínimo, ordenados por créditos de mayor a menor.
- **Repositorio:** **CursoRepository**
- **Service:** **CursoService**

```
// Repositorio:
```

```
// Service:
```

Ejercicio 14: Permisos de un rol (ManyToMany inversa)

- **Requerimiento:** Obtener todos los permisos asignados a un rol buscando por el **nombre** del rol (navegando la relación **roles** desde **Permiso**), sin distinguir mayúsculas.
- **Repositorio:** **PermisoRepository**
- **Service:** **PermisoService**

```
// Repositorio:
```

```
// Service:
```

Ejercicio 15: Estudiantes de un curso con **@Query** (JPQL y Native)

- **Requerimiento:** Obtener los estudiantes activos (**active = true**) matriculados en un curso (**cursoId**), ordenados por **apellido** ascendente.
 - Implementar una versión con **JPQL**.
 - Implementar una versión con **Native Query** (**nativeQuery = true**).
- **Repositorio:** **EstudianteRepository**
- **Service:** **EstudianteService**

```
// Repositorio (JPQL):
```

```
// Repositorio (Native SQL):
```

```
// Service:
```